A Native Swift LaTeX Rendering Engine

*Note: Sub-titles are not captured in Xplore and should not be used

1st Jane Smith
Dept. Computer Science
University of Technology
New York, USA
jsmith@university.edu

2nd Bob Jones
Research Lab
Institute for Rendering
San Francisco, USA
bjones@institute.org

Abstract—We present a fully native Swift implementation of a LaTeX rendering engine targeting Apple platforms. The engine uses CoreText and CoreGraphics to achieve typographic quality comparable to traditional TeX systems. We demonstrate accuracy across mathematical notation, two-column layouts, and tabular data, achieving a per-page SSIM score of 0.94 ± 0.02 against Overleaf-compiled reference output.

Index Terms—LaTeX, typesetting, Swift, CoreText, iOS, macOS

I. Introduction

The proliferation of mobile and tablet computing has created demand for offline scientific document authoring tools. Existing approaches either require network connectivity [1] or embed large binary TeX distributions that violate platform distribution guidelines.

Our contributions are threefold:

- A token-stream based parser supporting common LaTeX macro constructs.
- A full two-column layout engine with Knuth–Plass optimal line breaking.
- A math layout engine accurate to 0.1 pt for display-mode expressions.

II. Related Work

A. Server-Side Compilation

Overleaf [1] and similar services compile LaTeX remotely. This approach is feature-complete but requires a network connection, has latency of several seconds per compile, and raises privacy concerns for sensitive documents.

B. Native Renderers

Previous native implementations exist for iOS (iTeX, TeX Writer Pro) but achieve limited fidelity due to simplified line-breaking and absence of proper math layout.

III. System Architecture

The rendering pipeline consists of four stages, summarized in Table I. Fig. 1 shows a high-level overview of the architecture.

Fig. 1. Rendering pipeline overview. Input LaTeX source is tokenised, expanded, parsed into an AST, and finally typeset into CoreGraphics draw calls.

TABLE I
Rendering Pipeline Stages

Stage	Input	Output
Tokenizer	String	Token stream
Expander	Tokens	Expanded tokens
Parser	Expanded tokens	AST
Typesetter	AST	Typeset pages

A. Mathematical Typesetting

The math engine implements a subset of TeX box construction rules:

$$E = mc^2 \tag{1}$$

For multi-line aligned equations:

$$f(x) = x^2 + 2x + 1 \tag{2}$$

$$g(x) = \sqrt{x^2 + 1} \tag{3}$$

The axis height σ_{22} is set to $0.258 f_s$ where f_s is the design size, matching Computer Modern metrics.

IV. Evaluation

We evaluated the renderer on a corpus of 50 representative academic papers. Mean per-page SSIM compared to Overleaf output was 0.94 ± 0.02. The primary deficit was in complex TikZ figures (excluded from corpus).

V. Conclusion

We have demonstrated high-quality native LaTeX rendering on Apple platforms. Future work includes TikZ support and improved bibliography handling. Code available at https://github.com/example/typetex.

References

[1] Overleaf Team, "Overleaf: Real-time collaborative writing and publishing," https://www.overleaf.com, 2023.